

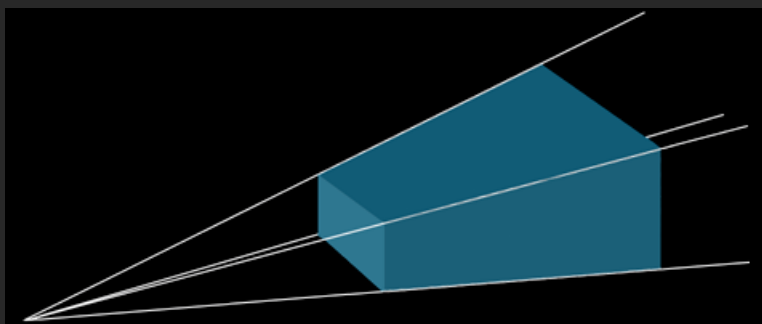
Inigo Quilez :: articles :: fixing frustum culling - 2013



Intro

There's not much to be said about view frustum culling. Or, is there? Well, it depends.

Most frustum culling implementations work just fine when the objects you are culling are small compared to the frustum they are being tested with. However, if your objects are big you might realize your engine is not as efficient anymore because, in fact, chances are you are doing frustum culling wrong...



A frustum defined by 6 planes

What most implementations do

If you are doing frustum culling the regular way (...), you are probably simply checking for your object (say, a bounding box or sphere) to be fully in the "outside" side of any of the 6 planes that define the frustum. If you are culling (bounding) boxes, then the thing will probably look like this:

```
// false if fully outside, true if inside or intersects
bool boxInFrustum( frustum3 const & fru, bound3 const & box )
{
    // check box outside/inside of frustum
    for( int i=0; i<6; i++ )
    {
        int out = 0;
        out += ((dot( fru.mPlane[i], vec4(box.mMinX, box.mMinY, box.mMinZ, 1.0f) )
        out += ((dot( fru.mPlane[i], vec4(box.mMaxX, box.mMinY, box.mMinZ, 1.0f) )
        out += ((dot( fru.mPlane[i], vec4(box.mMinX, box.mMaxY, box.mMinZ, 1.0f) )
```

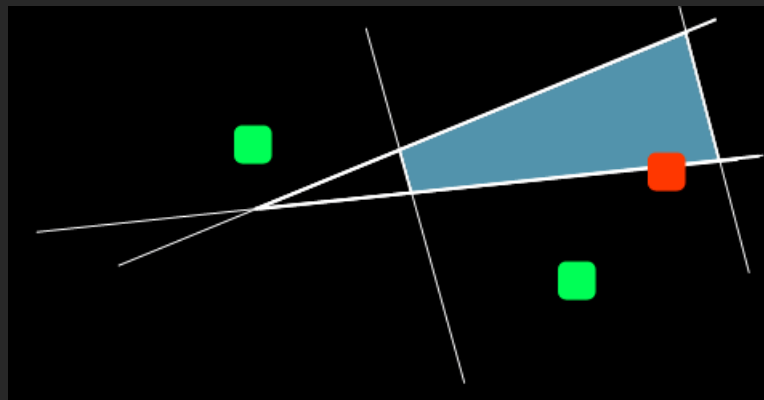
```

    out += ((dot( fru.mPlane[i], vec4(box.mMaxX, box.mMaxY, box.mMinZ, 1.0f) )
    out += ((dot( fru.mPlane[i], vec4(box.mMinX, box.mMinY, box.mMaxZ, 1.0f) )
    out += ((dot( fru.mPlane[i], vec4(box.mMaxX, box.mMinY, box.mMaxZ, 1.0f) )
    out += ((dot( fru.mPlane[i], vec4(box.mMinX, box.mMaxY, box.mMaxZ, 1.0f) )
    out += ((dot( fru.mPlane[i], vec4(box.mMaxX, box.mMaxY, box.mMaxZ, 1.0f) )
    if( out==8 ) return false;
}

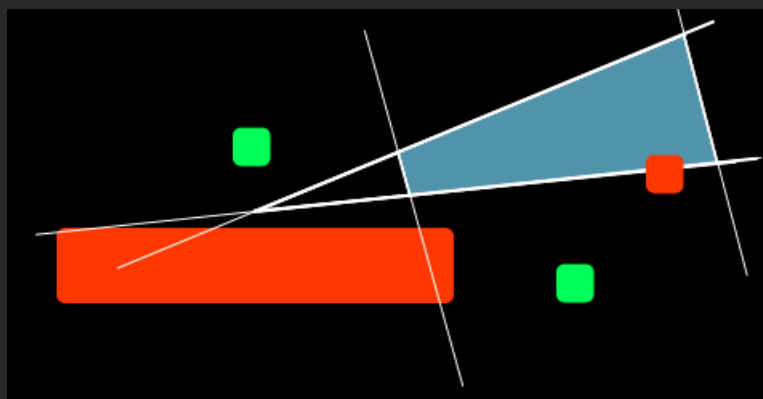
return true;
}

```

This code checks that the 8 corners of are to the same (external) side of any of the 6 planes, and if so, early exists indicating that the box does not overlap the frustum, and that it can be omitted from rendering or whatever task we are trying to optimize through the frustum culling process. This code works just fine in many situations, and does not report false negatives (not many, that is), so it is used blindly by many developers. However, if you are having an engine that is handling big scale objects (say huge chunks of planetary terrain, or huge quadtree nodes, or simply huge terrain meshes) you have probably noticed that this code simply doesn't work.



Regular frustum culling works just fine when the objects are small (green means the object can be culled, red means it can't)



Regular frustum failing to reject the big object behind the

*camera near plane, because it's so big it does
intersect some of the side planes of the frustum.*

What most implementations forget

The problem with big is that the changes of them intersecting some of the planes of the frustum are big as well. So even if an object is clearly not visible (see the big rectangle in the picture to the right), if it is big enough it will cross some of the frustum planes and perhaps not be fully in the exterior of any of them, and therefore the code above will fail to discard the object from further processing.

One simple solution to avoid this issue (or to make it less likely to happen) is to do an extra set of tests, by reversing the roles of the object and the frustum. For that, we need our frustum struct/class to contain the 8 corner points as well in addition to the 6 planes that define it. Then we can perform frustum-in-box tests in the exact same way as we did box against frustum: comparing the 8 vertices (of the frustum) against the 6 planes (of the box), and looking for an occurrence of a vertex being fully outside of any of the planes:

```
// false if fully outside, true if inside or intersects
bool boxInFrustum( frustum3 const & fru, bound3 const & box )
{
    // check box outside/inside of frustum
    for( int i=0; i<6; i++ )
    {
        int out = 0;
        out += ((dot( fru.mPlane[i], vec4(box.mMinX, box.mMinY, box.mMinZ, 1.0f) ) > 0.0f) ? 1 : 0);
        out += ((dot( fru.mPlane[i], vec4(box.mMaxX, box.mMinY, box.mMinZ, 1.0f) ) > 0.0f) ? 1 : 0);
        out += ((dot( fru.mPlane[i], vec4(box.mMinX, box.mMaxY, box.mMinZ, 1.0f) ) > 0.0f) ? 1 : 0);
        out += ((dot( fru.mPlane[i], vec4(box.mMaxX, box.mMaxY, box.mMinZ, 1.0f) ) > 0.0f) ? 1 : 0);
        out += ((dot( fru.mPlane[i], vec4(box.mMinX, box.mMinY, box.mMaxZ, 1.0f) ) > 0.0f) ? 1 : 0);
        out += ((dot( fru.mPlane[i], vec4(box.mMaxX, box.mMinY, box.mMaxZ, 1.0f) ) > 0.0f) ? 1 : 0);
        out += ((dot( fru.mPlane[i], vec4(box.mMinX, box.mMaxY, box.mMaxZ, 1.0f) ) > 0.0f) ? 1 : 0);
        out += ((dot( fru.mPlane[i], vec4(box.mMaxX, box.mMaxY, box.mMaxZ, 1.0f) ) > 0.0f) ? 1 : 0);
        if( out==8 ) return false;
    }

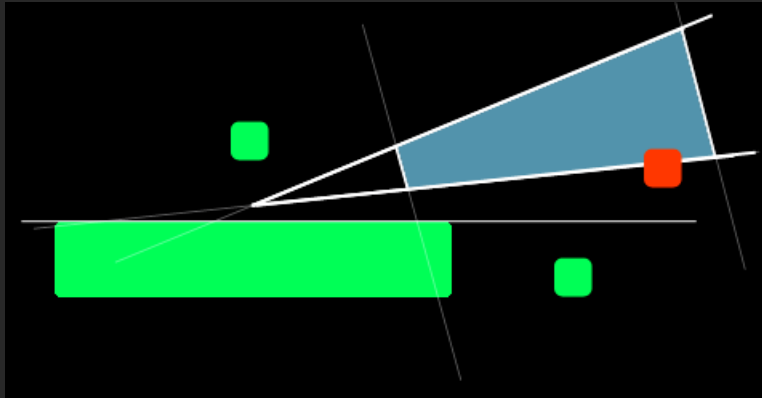
    // check frustum outside/inside box
    int out;
    out=0; for( int i=0; i<8; i++ ) out += ((fru.mPoints[i].x > box.mMaxX)?1:0);
    out=0; for( int i=0; i<8; i++ ) out += ((fru.mPoints[i].x < box.mMinX)?1:0);
    out=0; for( int i=0; i<8; i++ ) out += ((fru.mPoints[i].y > box.mMaxY)?1:0);
    out=0; for( int i=0; i<8; i++ ) out += ((fru.mPoints[i].y < box.mMinY)?1:0);
    out=0; for( int i=0; i<8; i++ ) out += ((fru.mPoints[i].z > box.mMaxZ)?1:0);
```

```

out=0; for( int i=0; i<8; i++ ) out += ((fru.mPoints[i].z < box.mMinZ)?1:0); :

return true;
}

```



Testing the frustum corners against the box planes helps solve the false overlap problem.

In the picture above, the 8 vertices of the frustum are all in the exterior side (above) of the top plane of the box. Hence the second block in `boxInFrustum()` will kick in and successfully report the box as not overlapping with the frustum.

These few extra lines of code, that unfortunately you don't find in most view frustum culling code out there, become crucial when doing rendering of big scale objects (terrains, planet octrees, etc), because such big objects do usually come with lots of processing for they are, indeed, big (think streaming of the data or procedural content generation associated with it). And for such simple and inexpensive addition to the frustum culling code, there's no excuse to not do it.

But we are not done yet

Actually, while the above implementation will work correctly for 99% of the use cases of view frustum culling, mathematically speaking there are still cases where the above cull might fail to detect non overlaps. In particular, if the frustum is not a view frustum, but some sort of very shallow/truncated frustum, boxes might be classified as intersecting when they aren't. Fortunately, this case won't happen since our frustums are usually constructed to be as long as we possibly can render, rather than shallow. Except in the cases of tile renderers where the view frustum culling is chopped in slices across depth and the resulting sub frustums' thickness are comparable to the boxes to be culled. In those cases, more complex frustum to box overlapping algorithms are needed for a mathematically perfect culling.

inigo quilez - learning computer graphics since 1994